Dataset info

- Isat normalized to 1 mm^2
- Starting with 2024 datarun only (DR2)
- Averaged from 10 to 20 ms
- isat on sweep probe is conditionally averaged for sweep voltages under a certain level

`Datasets_2023/PP1_isat_01`
- 10 ms time-averaged isat with machine settings and probe positions
- Validation dataruns: 08, 15, 33

`Datasets_2024/PP1_isat_01`
- 10 ms time-averaged isat with machine settings and probe positions
- Validation dataruns: 10, 31

`Datasets_combo/PP1_isat_01.npz`
- "`Datasets_2023/PP1_isat_01`" and "`Datasets_2024/PP1_isat_01`" combined

`Datasets_2023/PP1_isat_02`
- 10 ms time-averaged isat with machine settings and probe positions
- Validation dataruns (one more than 01): 08, 15, 23, 33

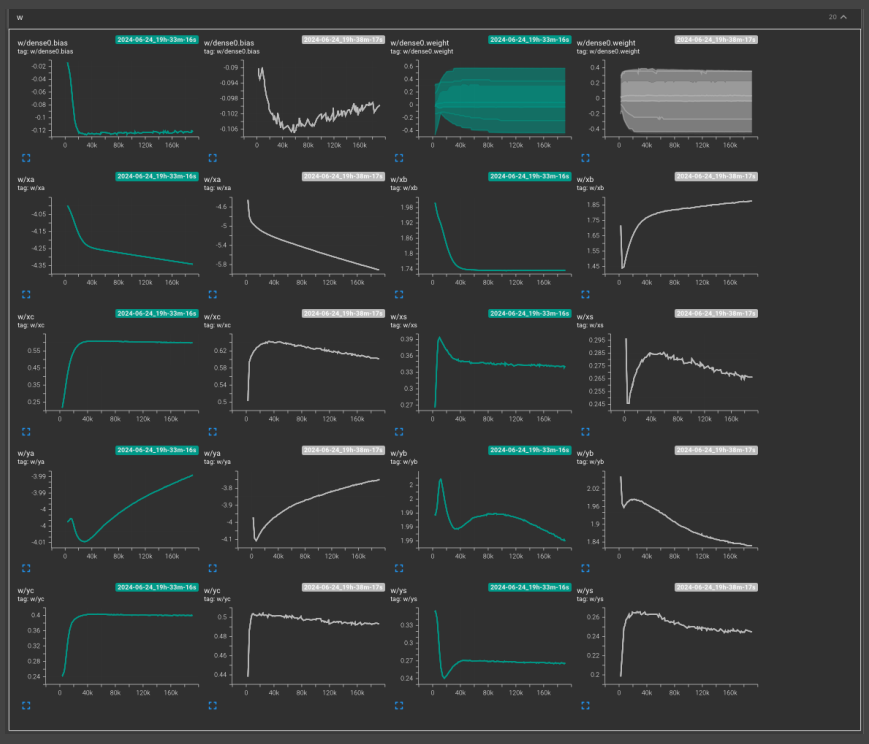
`Datasets_2024/PP1_isat_02`
- 10 ms time-averaged isat with machine settings and probe positions
- Validation dataruns (two more than 01): 02, 10, 19, 31

`Datasets_combo_PP1_isat_02`
- Combined 2023 and 2024 PP1_isat_02 series

Training runs

Dense
- still-vortex 34 (`2024-06-13_17h-50m-19s`)
  - base case. AdamW, 128 batch size, two layers of 128, tanh activations. Pretty good performance. Parameters: 18049
- absurd-salad-35 (`2024-06-13_18h-41m-20s`)
  - attempt at overfitting; 4 layers 512 each, tanh activation. Parameters: 794113
  - no overfitting after 500 (?) epochs

Linear runs
- Starting with simple SGD with momentum (beta = 0.5), learning rate 1e-4
  - 2024-06-24_19h-12m-37s
- `2024-06-24_19h-12m-37s`
  - Forgot shift parameter for abs (RMSE ~ 1.3e-1)
- `2024-06-24_19h-20m-24s`



Adam vs SGD

    - Forgot that I shouldn't add the tanh feature
- `2024-06-24_19h-26m-04s`
    - Gradients look a little small
- `2024-06-24_19h-33m-16s`
    - Adjusting LR to 1e-2
- `2024-06-24_19h-38m-17s`
    - Adam
- Adam vs SGD for simple models, the weights look similar (see image to the right)
    - both get to roughly the same RMSE (~1e-1)
- `2024-06-24_21h-14m-29s`
    - No tanh — straight up linear model. RMSE ~1.2e-1, just a straight line

Linear results: bad fits most of the time, but provides a worst-case sort of baseline. This problem / dataset appears to be highly nonlinear

Dense, DR combined

Pipeline validation by overfitting:
- `2024-07-09_17h-48m-48s`
    - Training on one batch
- `2024-07-09_18h-29m-31s`
    - Same, but saving losses
- `2024-07-09_18h-49m-11s`
    - Same, but reduced learning rate from 3e-4 to 1e-4 to try to avoid spikes (probably caused by large gradients)
- `2024-07-09_18h-56m-21s`
    - Zeroed input
    - Loss of 3.6e-2
- `2024-07-09_20h-37m-16s`
    - Back on training one batch, 1e-4 LR, but for 10k steps
- `2024-07-09_21h-21m-39s`
    - Same but batch size 8. Gets to 0 loss quite quickly. Funny enough, the testing loss is less than what the linear-like model gives

Evaluating losses and training performance
1 epoch unless specified
- `2024-07-10_12h-34m-55s`
    - 4 layers, 512 wide
- `2024-07-10_12h-38m-46s`
    - 4 layers, 1024 wide
- `2024-07-10_12h-39m-27s`
    - 8 layers, 512 wide
- `2024-07-10_14h-46m-12s`
    - 8 layers, 1024 wide
- `2024-07-10_14h-49m-45s`
    - 8 layers, 1024 wide, 10 epochs
- All the above have improve training and testing loss faster when model capacity was increased (either via layer depth or width — width seemed to have a larger effect)

- `2024-07-10_15h-18m-34s`
  - 12 layers, 2048 wide, 30 epochs
  - After about 6000 steps the train and test error hit what it would be on zeroed input, and the gradients vanished.
- `2024-07-10_15h-41m-52s`
  - This time let's try leaky ReLU
  - 12 layers, 2048 wide, 30 epochs
  - No overfitting evident
- `2024-07-10_16h-10m-24s`
  - 4 layers, 512 wide, 30 epochs, leaky ReLU
  - Trains faster and just as good as the 12x2048 when it comes to the testing loss
  - Qualitative performance seems a little worse than the bigger model (15h-41m-52s)

- There are only ~40 actual different dataruns. Checking with the validation dataset shows that the NN cannot interpolate that well between these 40 dataruns. The larger model seems to perform slightly better on this validation dataset, but I'm not certain why. The model trained on one batch actually has slightly better validation set loss.
  - Added 1 more validation datarun to DR1 and 2 more to DR2, for a total of 8 validation dataruns. 6 should be trained on, 2 should be extra special validation runs
  - Also corrected the offset in DR1 isat
- Validation runs were chosen to be somewhat spaced in time to cover machine variation (and should be somewhat random because of LHS sampling)

Testing on validation data with two holdout dataruns
- `2024-07-10_22h-18m-34s`
  - 4x512, 1k epochs, leaky ReLU
- `2024-07-10_22h-19m-35s`
  - 12x2048, 1k epochs, leaky ReLU
  - Get these massive spikes in all losses (large gradients). May need to try gradient clipping
  - Validation loss gets down to like 2.8e-3

- It's difficult to get the validation loss to crack around. 3e-3 The training loss got down to ~5e-6 and testing loss down to ~1.5e-5 on `2024-07-10_22h-18m-34s`. It appears to get the shape and general trends right though.
- There could be two reasons why this validation loss is so high:
  - 1. The network needs to see at least one example from a given LAPD configuration to know the magnitude/peak of the profile. This need implies that supplying a diagnostic like the discharge current may improve validation set performance.
  - 2. Along the lines of 1, but subtly different: the model doesn't know which neutral pressure regime we are in, and thus requires some isat samples. Supplying the neutral gas pressure (or in this gas, turbo-down flag) could rectify the issue. If this hypothesis is correct, training on a particular run set (DR1 or DR2, but not both simultaneously) should have better validation performance. This solution is the obvious one to try first.

- `2024-07-11_09h-37m-17s`
  - Training only on DR1, gradients now clipped. 100 epochs
  - Validation set loss (DR1 only) ends up at around 6.6e-3 at the end
  - Final non-best checkpoint (99-369), DR1 validation loss was 6.4e-3, DR2 was 1.2e-1
- `2024-07-11_09h-37m-36s`
  - Training only on DR2, gradients now clipped
  - Validation set loss ends up at around 7.1e-3 at the end
  - On final non-best checkpoint (99-368), DR2 validation loss was 5.4e-3, DR1 was 2.6e-2

- Training on just one datarun gets even worse validation loss, even when just isolated to the validation set it was trained on

Verifying if low validation loss is caused by insufficient diversity
—> loss should go up with fewer dataruns

- `2024-07-11_14h-23m-11s`, `2024-07-11_14h-32m-45s` are the full training set (ignore the dataruns_exclude hyperparam) — deleted from wandb
- `2024-07-11_18h-28m-44s`
  - Before: training on 59 runs, this one is 49
  - Excluding: DR1_09 DR1_14 DR1_22 DR1_35 DR1_36 DR2_04 DR2_11 DR2_12 DR2_14 DR2_28
- `2024-07-11_18h-29m-25s`
  - Training on 39 runs
  - Excluding: DR1_04 DR1_09 DR1_14 DR1_17 DR1_20 DR1_22 DR1_30 DR1_35 DR1_36 DR1_39 DR2_04 DR2_08 DR2_11 DR2_12 DR2_13 DR2_14 DR2_17 DR2_18 DR2_20 DR2_28
- `2024-07-11_18h-30m-59s`
  - Training on 29 runs
  - Excluding: DR1_02 DR1_04 DR1_06 DR1_09 DR1_14 DR1_17 DR1_20 DR1_22 DR1_26 DR1_28 DR1_30 DR1_35 DR1_36 DR1_37 DR1_39 DR2_03 DR2_04 DR2_08 DR2_11 DR2_12 DR2_13 DR2_14 DR2_17 DR2_18 DR2_20 DR2_21 DR2_23 DR2_28 DR2_29 DR2_30
- `2024-07-11_18h-32m-08s`
  - Training on 19 runs
  - Excluding: DR1_01 DR1_02 DR1_04 DR1_06 DR1_07 DR1_09 DR1_10 DR1_13 DR1_14 DR1_17 DR1_18 DR1_20 DR1_22 DR1_26 DR1_28 DR1_30 DR1_35 DR1_36 DR1_37 DR1_39 DR2_03 DR2_04 DR2_05 DR2_08 DR2_11 DR2_12 DR2_13 DR2_14 DR2_15 DR2_17 DR2_18 DR2_20 DR2_21 DR2_22 DR2_23 DR2_26 DR2_27 DR2_28 DR2_29 DR2_30
- `2024-07-11_18h-35m-53s`
  - Training on 9 runs
  - Excluding: DR1_01 DR1_02 DR1_04 DR1_06 DR1_07 DR1_09 DR1_10 DR1_13 DR1_14 DR1_17 DR1_18 DR1_20 DR1_21 DR1_22 DR1_25 DR1_26 DR1_28 DR1_29 DR1_30 DR1_34 DR1_35 DR1_36 DR1_37 DR1_38 DR1_39 DR2_01 DR2_03 DR2_04 DR2_05 DR2_06 DR2_08 DR2_09 DR2_11 DR2_12 DR2_13 DR2_14 DR2_15 DR2_17 DR2_18 DR2_20 DR2_21 DR2_22 DR2_23 DR2_24 DR2_25 DR2_26 DR2_27 DR2_28 DR2_29 DR2_30
- `ensemble_test`
  - `2024-07-11_18h-38m-32s`
  - `2024-07-11_18h-44m-22s`
  - `2024-07-11_18h-44m-33s`
  - `2024-07-11_18h-47m-18s`
  - `2024-07-11_18h-47m-32s`
  - Training on all 59
  - Seeds 42, 0, 1, 2, 3
- `2024-07-12_15h-01m-37s`
  - Batch data visualization test
- `2024-07-13_16h-41m-40s`
  - Pressure flag test
- `2024-07-13_16h-42m-27s`
  - Pressure flag test, 100 epochs
  - Validation loss is much better, seems to bottom out < 14 epochs in
- `ens_DRcombo_03_cv-0`

    - Ensemble on the 0th cross-validation dataset; 30 epochs, 7 different seeds
- `ens_DRcombo_03_cv-all`
    - Collection of runs over all 7 cross-validation datasets; 30 epochs, 7 different seeds

- `2024-07-17_13h-57m-57s`
    - Testing NLL loss (predicting mean and variance)
    - The MSE on the validation set was 0.001953 (!)
    - The prediction of the variance doesn't seem to correlate with error
- `2024-07-17_15h-04m-04s`
    - Same as the one above but saving a checkpoint ever 50 seconds to see how much the validation error changes with training

Model	Train err	Valid err	Train std	Valid std
model-11-551	0.00015038828	0.0022607094	0.010953	0.015891585
model-23-232	5.3848853E-05	0.0019495541	0.005412553	0.006602611
model-34-510	5.126938E-05	0.0018295944	0.005704496	0.0062265587
model-44-459	4.9215578E-05	0.002100382	0.0049253926	0.0051003066
model-53-724	5.8530728E-05	0.0017580009	0.0038966043	0.003950303
model-62-613	4.0731607E-05	0.0020206962	0.0047225943	0.0047209905
model-71-190	3.623333E-05	0.0018899719	0.0034727866	0.0037843161
model-78-598	3.3115648E-05	0.0019563155	0.004282812	0.0043779393
model-86-143	2.7823775E-05	0.001945922	0.00378215	0.004540547
model-93-57	3.368442E-05	0.002028984	0.004417203	0.0052141347
model-99-446	2.8870872E-05	0.0019540095	0.004984819	0.00579572

    - Validation error seems to not change much past 30 epochs, so let's use that for the ensemble
- `ens_NLL_DRCombo-03_cv-0`
    - An ensemble of models using the NLL loss function to get aleatoric and epistemic uncertainty
    - Seems to work pretty well. Can break down by aleatoric and epistemic uncertainty
    - z-scores are much worse on validation dataset
- percentile-based grad norm clipping slows down training because the list gets large. Fixed
- `hyper_weight-decay`
    - Testing 17 different weight decays to find the optimal amount for good calibration on the validation set. A weight decay of
- `2024-07-18_22h-34m-17s`
    - Trying using the radius instead of x and y
    - This didn't work because the mean-ptp scaling breaks the geometric relationship — this needs to be taken care of in the dataset
- `2024-07-18_22h-49m-10s`
    - Modifying the dataset this time
    - The y coordinate was added by x mean instead of y mean — need to retrain because the info is (consistently) wrong
    - The train, test, and validation MSE are considerably higher than the, no-weight-decay models
- `2024-07-18_23h-17m-09s`

    - Retraining with correct data implementation
    - Model isn't well calibrated (validation set does not predict it's own uncertainty) but the predictions are… smoother
- `2024-07-18_23h-43m-34s`
    - Same but with weight decay of 10.0
    - Moved this in the radial_weight-decay-scan ensemble folder
- `2024-07-19_15h-39m-15s`
    - Same but with weight decay 1.0
    - Moved this in the radial_weight-decay-scan ensemble folder

- Randomly checked y and z positions on NLL ensemble runs — looks fine to me.

- `radial_weight-decay-scan`
    - 12 scans with the radial coordinate model, 0.001 to 30.0
    - Also added earlier evaluations of 1.0 and 10.0 to the folder
    - Accidentally ran the same weight decays several times

- Made dataset (04) with a top gas puff flag as well

- `2024-07-23_16h-52m-01s`
    - Test on dataset 04 (no weight decay, just NLL loss)
    - It's a poorly-calibrated model but it gets pretty decent validation error (1.8e-3 averaged over all shots)

- `wd-scan-[WD]_big`
    - 4 different weight decay values (WD): 0.01, 0.1, 1.0, and 10.0
    - Dataset 04 cv-0
    - Each model is an ensemble of 5
    - 8 layers, 1024 wide, 500 epochs, lr 3e-4
    - many models are unstable, affecting final performance
    - When calculating the differences in error, I needed to use 64 bit floats instead of 32 to avoid sqrt(< 0) errors.

- `wd-1.0_big-cv-[CV]`
    - scanning across cross validation datasets, taking model params from `wd-scan-1.0_big`
    - aaa

- `2024-08-29_15h-16m-16s`
- `2024-08-29_15h-19m-10s`
- `2024-08-29_15h-25m-44s`
    - Beta-NLL loss (with a value of 0.5)
    - Testing gradient norm clipping. First is 1e5, second is 1e1, and third is 1e0
    - Beta-NLL seems to be a sufficient regularizer that clipping may not be needed
- `2024-08-29_17h-19m-38s`
    - Beta-NLL loss with a value of 0 for reference
- Beta-NLL loss seems to slightly improve validation set performance and is slightly better calibrated

- `2024-08-29_18h-44m-09s`
    - Trying learning rate decay 1/(1 + epoch/70), beta-NLL 0.5
- `2024-08-30_11h-26m-21s`
    - Trying Exponential learning rate decay e^(-epoch/140)

- `2024-08-30_11h-39m-15s`
  - Trying 1/sqrt(epoch-70) if epoch>70 learning rate
- Of these three schedules, the epoch^-1 performed better than exponential (but not by much) on the validation loss. The 1/sqrt was a little worse. The opposite was true for training loss
- No scheduling performed worse in all cases. The takeaway here seems to be that scheduling is good to have, but the method may not matter too much
- Exponential might perform better when training for longer
- epoch^-1 had the lowest amount of variability among batches
- epoch^-1 is the best calibrated but the constant LR is much worse calibrated

- `2024-08-30_12h-58m-07s`
  - 1/epoch LR decay, 0 beta-NLL, gradient clip norm to 1

- `beta-NLL_wd-[WD]`
  - Weight decay scan of [0.01, 0.1, 1.0, 2.0, 4.0, 6.0, 8.0, 10.0, 20.0, 30.0, 40.0, 50.0]
  - Using beta-NLL loss with a beta value of 0.5
  - Using 1/1+epoch learning rate decay

- `beta_NLL_SiLU`
  - 5-ensemble, same as `beta-NLL_wd-0.1` but with SiLU activations
  - Seems to struggle a bit on validation loss — it wanders all over the place

- `2024-09-05_18h-39m-20s`
- `2024-09-05_18h-43m-18s`
- `2024-09-05_18h-48m-23s`
  - Trying Tanh activations with 0.1, 0.01, and 0.0 weight decays. Validation loss seems to be sensitive to it and it wanders while training. Kind of strange
  - All runs killed before or around 100 epochs

9/16: My Thomson error calculation had a glitch for DR1. Resaved. DR2 looks good.

- `beta-NLL_dataset-05`
  - Typical ReLU model same as the to the `beta-NLL_wd-[WD]` models
  - Ensemble where the y coordinates of DR2 were adjusted. Gets similar errors/losses across the board (but the validation set looks a bit worse, but that could just be me)
  - x-y asymmetry appears to be improved when looking at predicted x and y profiles

- `beta-NLL_dataset-06`
  - Same as `-05`, but with both DR1 and DR2 adjusted along the y-axis

Next time:
- Try to account for probe centering errors — clean data
- Fine tune calibration factors on test set?
- Remove probe angle, add radius feature
- Use ensemble of cross-validated model (or total dataset)

EBM ideas:
- Train on MSI + this dataset to try to get mega-diverse coverage; may be able to compensate for gaps (model learns from diodes+current+ifo (maybe) to predict isat)
- Train on simple simulation data to try to extrapolate to unseen conditions
- Make fake data for zero isat at r=50cm